

**Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”**

Розрахунково-графічна робота

**Порівняння алгоритмів ройового інтелекту
(ACO, GA, Tabu Search)
для задачі маршрутизації транспортних засобів
з часовими вікнами**

Виконав: студент гр.Фі-21
Грунда Я.І.

Перевірив: ас. Железняков Д.В.

Київ — 2025

Зміст

1	Актуальність теми	2
2	Мета та завдання роботи	2
3	Постановка задачі VRPTW	2
3.1	Параметри штрафів	3
4	Опис алгоритму Ant Colony Optimization	3
4.1	Загальна ідея АСО	3
4.2	Деталі реалізованої модифікації	4
4.2.1	Структури даних	4
4.2.2	Параметри алгоритму	4
4.2.3	Крок побудови туру	4
4.2.4	Глобальне оновлення феромону	5
4.2.5	Критерій зупинки	5
4.2.6	Запис історії	5
4.2.7	Повернене значення	5
5	Опис алгоритму Genetic Algorithm (GA)	5
5.1	Загальна ідея GA	5
5.2	Деталі реалізованої модифікації	6
5.2.1	Структури даних	6
5.2.2	Параметри алгоритму	6
5.2.3	Оператори	6
5.2.4	Критерій зупинки	7
5.2.5	Запис історії	7
5.2.6	Повернене значення	7
6	Опис алгоритму Tabu Search	7
6.1	Загальна ідея Tabu Search	7
6.2	Деталі реалізованої модифікації	8
6.2.1	Структури даних	8
6.2.2	Параметри алгоритму	8
6.2.3	Сусідство	8
6.2.4	Алгоритм однієї ітерації	8
6.2.5	Запис історії	8
6.2.6	Критерій зупинки	9
6.2.7	Повернене значення	9
7	Експериментальна частина	9
7.1	Налаштування тестів	9
7.2	Опис інстансів Solomon	9
7.3	Результати	10
8	Деякі графіки збіжності та маршрутів(інстанс r101)	11
9	Висновки	17

1. Актуальність теми

Транспортні витрати становлять до 60 % операційного бюджету більшості логістичних компаній. Рациональне планування маршрутів із урахуванням часових вікон доставки (*Vehicle Routing Problem with Time Windows*, VRPTW) дає змогу скоротити пробіг, кількість залучених автомобілів і, відповідно, зменшити витрати пального та викиди CO₂. Серед метаверстик, що практично застосовуються, часто розглядають алгоритми ройового інтелекту (Ant Colony Optimization, Genetic Algorithm) та траєкторні методи (Tabu Search). Тому доцільно оцінити продуктивність цих алгоритмів на вибірці тестових інстансів VRPTW.

2. Мета та завдання роботи

Мета: порівняти якість ACO, GA та Tabu Search на еталонних інстансах Solomon VRPTW.

Завдання:

1. Вивчити математичну постановку VRPTW.
2. Реалізувати версії ACO, GA, Tabu Search з однаковою цільовою функцією.
3. Розробити інструмент автоматичного запуску та логування історії cost.
4. Провести серію експериментів з однаковим тайм-лімітом.
5. Порівняти мінімальні, середні та максимальні показники cost і distance, зробити висновки.

3. Постановка задачі VRPTW

Нехай задано орієнтований граф $G = (V, E)$, де $V = \{0, 1, \dots, n\}$: вершина 0 — депо, а $1..n$ — клієнти. Дуга $(i, j) \in E$ характеризується відстанню d_{ij} (дорівнює часу проїзду). Для кожного клієнта i ($i \geq 1$) задано

- попит q_i ;
- часовий інтервал $[e_i, l_i]$, у межах якого має розпочатися обслуговування;
- тривалість сервісу s_i .

У депо знаходиться флот із $K_{\max}$ однакових вантажівок ємністю Q та власним часовим вікном $[e_0, l_0]$. Необхідно побудувати набір маршрутів $\mathcal{R} = \{R_1, \dots, R_K\}$ ($K \leq K_{\max}$), кожний з яких починається й закінчується в депо, так щоб

1. кожен клієнт відвіданий рівно один раз;
2. вантажні та часові обмеження виконуються (або штрафуються) для кожного маршруту;
3. мінімізується наступна цільова функція:

$$\min D + \lambda_{\text{late}} \sum_{i=1}^n L_i + \lambda_{\text{over}} \sum_{k=1}^K O_k + \lambda_{\text{depot}} \sum_{k=1}^K D_k + \lambda_{\text{veh}} \max(0, K - K_{\text{max}}),$$

де

D — сума всіх відстаней по маршрутах;

L_i — **запізнення** на клієнті i : $L_i = \max(0, T_i - l_i)$, T_i — фактичний час початку сервісу;

O_k — **перевантаження** маршруту k : $O_k = \max(0, \sum_{i \in R_k} q_i - Q)$;

D_k — **порушення вікна депо** для маршруту k : $D_k = \max(0, t_k^{\text{back}} - l_0)$, де t_k^{back} — час повернення у депо;

λ_* — великі штрафні коефіцієнти, що збігаються з `lambda_late`, `lambda_over`, `lambda_depot`, `lambda_veh` у функції `evaluate_solution`.

Таким чином, якщо всі обмеження виконуються, штрафні суми L_i , O_k , D_k дорівнюють нулю, і функція зводиться до мінімізації маршрутової відстані D та кількості використаних машин.

3.1. Параметри штрафів

Коефіцієнт	Значення	Коментар
λ_{over}	1 000	заборона перевантажень авто
λ_{late}	1 000	заборона запізнень до клієнтів
λ_{depot}	1 000	заборона пізнього повернення до депо
λ_{veh}	10 000	жорстке обмеження кількості авто

Табл. 1: Налаштування цільової функції

4. Опис алгоритму Ant Colony Optimization

4.1. Загальна ідея АСО

Ant Colony Optimization (ACO) належить до метаевристических ройового інтелекту і моделює колективну поведінку колонії мурах, що шляхом відкладання *феромону* знаходять найкоротші стежки до джерела їжі. На рівні алгоритму це відбувається так:

1. Кожен агент (“мураха”) послідовно будує розв’язок, переходячи зі вершини i у вершину j із вірогідністю, пропорційною $\tau_{ij}^\alpha \eta_{ij}^\beta$, де τ_{ij} — інтенсивність феромону, η_{ij} — статична евристика (для VRPTW — $1/d_{ij}$).

2. Після побудови всіх розв’язків:

- феромон *випаровується* $\tau_{ij} \leftarrow (1 - \rho)\tau_{ij}$;

- кращі розв'язки *накопичують* феромон $\tau_{ij} \leftarrow \tau_{ij} + \Delta\tau_{ij}$, підсилюючи ребра, якими варто ходити.
3. Процедура повторюється, доки не буде вичерпано ліміт ітерацій або часу. У процесі мурахи дедалі частіше використовують ребра, що ведуть до глобально кращих траскторій.

Таким чином АСО комбінує **різноманітність** (рандомізовані ходи) та **інтенсифікацію** (підсилення кращих ребер) і поступово концентрує пошук поблизу найякісніших рішень.

4.2. Деталі реалізованої модифікації

4.2.1. Структури даних

- $\tau \in \mathbb{R}^{(n+1) \times (n+1)}$ — матриця феромону, ініціалізована константою τ_0 .
- $\eta_{ij} = 1/(d_{ij} + 10^{-6})$ — евристична привабливість.
- `history`: список кортежів (`iter`, `t`, `best_cost`, `best_dist`) для графіків збіжності.

4.2.2. Параметри алгоритму

Параметр	Значення	Значення за замовчуванням
Кількість мурах n_{ants}	10 ... 50	50
α (вага феромону)	1–2	1.0
β (вага евристики)	2–4	3.0
ρ (випаровування)	0.1–0.3	0.1
q_0 (ймовірність жадібного кроку)	0.3–0.7	0.6
τ_0 (старт. феромон)	$10^{-4} \dots 10^{-2}$	10^{-3}
<code>elitist</code>	True/False	True

Табл. 2: Ключові параметри реалізації АСО

4.2.3. Крок побудови туру

1. Старт у депо 0; множина невідвіданих клієнтів U .
2. Для поточного вузла i обчислити $\text{desir}_j = \tau_{ij}^\alpha \eta_{ij}^\beta$ для всіх $j \in U$.
3. З імовірністю q_0 вибрати $j = \arg \max_{j \in U} \text{desir}_j$; інакше провести рулетку по desir_j .
4. Додати j до туру, виконати *локальне* зменшення феромону: $\tau_{ij} \leftarrow (1 - \rho)\tau_{ij} + \rho\tau_0$.
5. Повторювати, поки $U = \emptyset$; на завершення виконати `local-update` для ребра останній $\rightarrow$ депо.

4.2.4. Глобальне оновлення феромону

Після кожної ітерації:

$$\tau_{ij} \leftarrow (1 - \rho)\tau_{ij} + \sum_{(i,j) \in R_*} \frac{1}{\text{cost}(R_*)},$$

де R_* — глобально найкращий тур (режим `elitist`). У не-`elitist`-режимі сума береться по всіх мурах поточної ітерації.

4.2.5. Критерій зупинки

Алгоритм працює, доки одночасно виконується

$$\text{elapsed} < \text{time_limit} \quad \wedge \quad \text{iter} < \text{max_iter}.$$

Обидва параметри задаються користувачем; кожен може бути 'None'.

4.2.6. Запис історії

Після кожної ітерації у `history` додається запис $(k, t, \text{best_cost}, \text{best_dist})$, що дозволяє будувати графіки «best-cost / distance vs time» і порівнювати збіжність з іншими алгоритмами.

4.2.7. Повернене значення

Метод `run()` віддає пару

$$(\text{best_cost}, \text{best_tour}),$$

де `best_tour` — `giant-tour` (перестановка клієнтів), а вартість `best_cost` враховує відстань та всі штрафи, визначені у функції `evaluate_solution`.

5. Опис алгоритму Genetic Algorithm (GA)

5.1. Загальна ідея GA

Генетичний алгоритм (GA) — це популяційна метаевристика, що імітує еволюційні процеси: *відбір найкращих*, *кросовер* (рекомбінація ДНК) та *мутацію*. У контексті VRPTW:

1. Кожен індивід кодує **giant-tour** — перестановку клієнтів $1..n$; депо 0 додається під час декодування.
2. Популяція розміром P оцінюється цільовою функцією (відстань + штрафи за запізнення, перевантаження тощо).
3. На кожному поколінні
 - (a) обираємо батьків оператором турнірного відбору;
 - (b) створюємо потомка кросовером OX (Order Crossover);
 - (c) випадково мутуємо потімка, міняючи місцями 2–3 гени;
 - (d) еліта (найкращий індивід) копіюється без змін.

- Після завершення ліміту часу або заданої кількості поколінь повертається глобальний найкращий розв’язок.

Таким чином GA забезпечує **різноманітність** за рахунок мутацій і рекомбінацій, а **інтенсифікацію** — через відбір кращих та елітизм.

5.2. Деталі реалізованої модифікації

5.2.1. Структури даних

- `Individual = (tour, cost)`, де `tour` — список клієнтів, `cost` — значення цільової функції.
- `pop` — список із P особин.
- `best` — глобально найкращий індивід.
- `history` — кортежі (покоління, t , `best_cost`, `best_dist`).

5.2.2. Параметри алгоритму

Параметр	Діапазон / сенс	Значення за замовчуванням
P (<code>pop_size</code>)	100–500	300
Ставка кросоверу p_{cx}	0.7–0.95	0.9
Ставка мутації p_{mut}	0.05–0.3	0.2
Розмір турніру k	2–5	3
Еліта	1 індивід	+1

Табл. 3: Ключові параметри GA-реалізації

5.2.3. Оператори

Відбір k -арний турнір без повернення: обираємо k випадкових індивідів, беремо того, чия `cost` мінімальна.

Кросовер — *Order Crossover (OX)*. Зберігає відносний порядок клієнтів:

- випадковий сегмент $[a, b)$ копіюється з батька 1;
- решту позицій послідовно заповнюють гени батька 2, зберігаючи їхню відносну чергу.

Мутація — *swap*. З імовірністю p_{mut} міняємо місцями 2–3 довільні позиції.

Елітизм Найкращий індивід популяції копіюється у нове покоління без змін.

5.2.4. Критерій зупинки

Цикл триває, доки виконується

$$\text{elapsed} < \text{time_limit} \quad \wedge \quad \text{generation} < \text{max_gen}.$$

Будь-який з параметрів може бути `None`.

5.2.5. Запис історії

Наприкінці кожного покоління:

$$\text{history} += (g, t, \text{best_cost}, \text{best_dist}),$$

що дозволяє порівнювати криві збіжності з ACO та Tabu Search.

5.2.6. Повернене значення

`run()` повертає

$$(\text{best_cost}, \text{best_tour}),$$

де `best_tour` — giant-tour із мінімальним `cost` за всю еволюцію.

6. Опис алгоритму Tabu Search

6.1. Загальна ідея Tabu Search

Tabu Search (TS) — *траєкторна* метаевристика: вона оперує одним робочим розв’язком і крок-за-кроком рухається його околom. Основні принципи:

- **Найкращий доступний сусід.** На ітерації формується підмножина можливих рухів (swap/relocate). Вибирають той, чия вартість мінімальна, навіть якщо вона *гірша* за поточну. Так пошук може «перестрибувати» локальні мінімуми.
- **Короткострокова пам’ять (tabu-list).** Застосований рух заносять у список табу довжини *tenure* і тимчасово забороняють — це запобігає негайному поверненню та циклам.
- **Аспірація.** Якщо табу-рух генерує розв’язок, який *покращує* глобальний *best*, заборона ігнорується. Отже, «хороші винятки» не блокуються.
- **Довгострокова еволюція.** Чергуючи керовані погіршення і заборони, алгоритм поступово досліджує нові області простору й концентрується навколо перспективних зон.

Такий баланс *інтенсифікації* (пошук найкращого сусіда) і *диверсифікації* (табування та аспірація) дозволяє TS ефективно обходити локальні пастки та знаходити якісні рішення VRPTW.

1. Початкова точка — випадковий giant-tour.
2. На кожній ітерації генерується підмножина сусідів (swap/relocate), оцінюються їхня вартість, і вибирається ліпший *допустимий* кандидат.

3. Виконаний рух заноситься в табу-список довжини `tenure`; прострочені записи автоматично видаляються.
4. Алгоритм завершується після тайм-ліміту або коли досягнуто `max_iter`.

6.2. Деталі реалізованої модифікації

6.2.1. Структури даних

- **Move** = (i, j, type) , де $\text{type} \in \{\text{"swap"}, \text{"reloc"}\}$.
- `tabu` — черга записів $(\text{move}, \text{expiry})$.
- `history` — $(\text{iter}, t, \text{best_cost}, \text{best_dist})$.

6.2.2. Параметри алгоритму

Параметр	Діапазон / сенс	Значення за замовчуванням
Довжина <code>tabu tenure</code>	10–40	20
Розмір підмножини округи <code>neigh_size</code>	100–500	200
Частка <code>swap</code> у сусідах <code>swap_rate</code>	0.3–0.7	0.5

Табл. 4: Ключові параметри Tabu Search

6.2.3. Сусідство

- **Swap** (i, j) : міняє місцями позиції i і j у `giant-tour`.
- **Relocate** $(i \rightarrow j)$: виймає клієнта з позиції i і вставляє перед j (або після, щоб зберегти індекси).

На кожній ітерації випадково відбирається `neigh_size` рухів згідно зі `swap_rate`.

6.2.4. Алгоритм однієї ітерації

1. Для кожного кандидата c обчислюємо $\text{cost}(c) = \text{evaluate_solution}$.
2. Відкидаємо рух `tabu`, якщо він не дає кращий `global best` (аспірація).
3. Обираємо кандидата з мінімальним `cost`; робимо його новим “current”.
4. Якщо $\text{cost} < \text{best_cost}$ — оновлюємо глобальний `best`.
5. Додаємо рух у `tabu` зі строком дії $\text{iter} + \text{tenure}$.

6.2.5. Запис історії

Після кожної ітерації:

`history += (it, t, best_cost, best_dist)`.

6.2.6. Критерій зупинки

Пошук триває, доки

$$\text{elapsed} < \text{time_limit} \quad \wedge \quad \text{iter} < \text{max_iter}.$$

6.2.7. Повернене значення

Метод `run()` повертає

$$(\text{best_cost}, \text{best_tour}),$$

де `best_tour` — giant-tour з мінімальним штрафним `cost`, знайдений за весь час роботи.

7. Експериментальна частина

7.1. Налаштування тестів

Тайм-ліміт кожного запуску — 60 с. Для інстансу **C101** виконано 10 запусків, для решти — по 5 запусків з різними початковими `seed`.

Табл. 5: Параметри запуску кожного алгоритму

Параметр	ACO	GA	Tabu Search
Популяція / мурахи	$n_{\text{ants}} = 25$	$P = 25$	—
Вага феромону α	1.0	—	—
Вага евристики β	1.5	—	—
Коеф. випаровування ρ	0.1	—	—
q_0 (жадібність)	0.3	—	—
Початковий феромон τ_0	1×10^{-3}	—	—
Elitist-режим	True	—	—
Турнір k	—	5	—
Ставка кросоверу p_{cx}	—	0.9	—
Ставка мутації p_{mut}	—	0.8	—
Tenure табу-списку	—	—	15
Розмір околу neigh_size	—	—	250

7.2. Опис інстансів Solomon

Інстанси Solomon VRPTW поділяються на три класи:

- **С-серія (Clustered)** — клієнти згруповані у декілька щільних кластерів; корисні для перевірки алгоритму на «багато коротких рейсів усередині кластерів».
- **Р-серія (Random)** — координати клієнтів рівномірно розкидані по всій області; дають довгі радіальні маршрути.
- **РС-серія (Random-Clustered)** — змішаний тип, деякі точки утворюють кластери, решта — розкидані випадково.

У кожній серії є підсерії **1** та **2**: C1, R1, RC1 мають *вузькі* часові вікна, через що потрібна більша кількість машин; серії C2, R2, RC2 мають *ширші* вікна й зазвичай дозволяють менше транспортних засобів.

7.3. Результати

Алгоритм	min cost	avg cost	max cost	min dist	avg dist	max dist
ACO	2491404.42	2534961.06	2584992.51	1889.96	1949.43	2048.73
GA	3171845.28	4155713.68	5140755.60	3480.26	3672.15	3955.61
Tabu	586228.99	1208464.63	1435677.21	2655.69	2986.31	3410.03

Табл. 6: Підсумкова статистика (інстанс c101)

Алгоритм	min cost	avg cost	max cost	min dist	avg dist	max dist
ACO	2423103.44	2469665.94	2500398.61	1901.07	1941.11	1963.06
GA	69401.78	118311.21	161616.55	2795.04	2920.85	3073.03
Tabu	1919.95	1946.90	1967.86	1919.95	1946.90	1967.86

Табл. 7: Підсумкова статистика (інстанс r101)

Алгоритм	min cost	avg cost	max cost	min dist	avg dist	max dist
ACO	27213089.16	28124545.31	29333216.11	1091.34	1179.79	1345.40
GA	7935999.73	10427469.54	13690801.96	3066.02	3179.34	3311.68
Tabu	1879423.91	2734560.07	4135349.99	1500.87	1738.27	2080.15

Табл. 8: Підсумкова статистика (інстанс c201)

Алгоритм	min cost	avg cost	max cost	min dist	avg dist	max dist
ACO	18066407.52	19122801.23	19818220.15	1276.07	1402.66	1557.06
GA	904598.66	1556908.51	1970099.22	2724.61	2854.76	2984.61
Tabu	802890.86	1051323.29	1294973.43	1932.53	2229.28	2462.77

Табл. 9: Підсумкова статистика (інстанс r201)

Алгоритм	min cost	avg cost	max cost	min dist	avg dist	max dist
ACO	1883688.28	1931197.18	1978589.90	2057.32	2220.33	2599.46
GA	48706.48	60048.44	74358.90	3356.56	3414.44	3508.37
Tabu	1949.75	2052.95	2139.47	1949.75	2052.95	2139.47

Табл. 10: Підсумкова статистика (інстанс rc101)

Алгоритм	min cost	avg cost	max cost	min dist	avg dist	max dist
ACO	16937688.93	17538615.63	18146784.69	1124.59	1221.99	1369.29
GA	826397.00	1140599.06	1724271.78	3534.04	3721.18	4147.99
Tabu	411164.46	598196.68	977916.01	2731.54	2829.81	2966.60

Табл. 11: Підсумкова статистика (інстанс rc201)

8. Деякі графіки збіжності та маршрутів(інстанс r101)

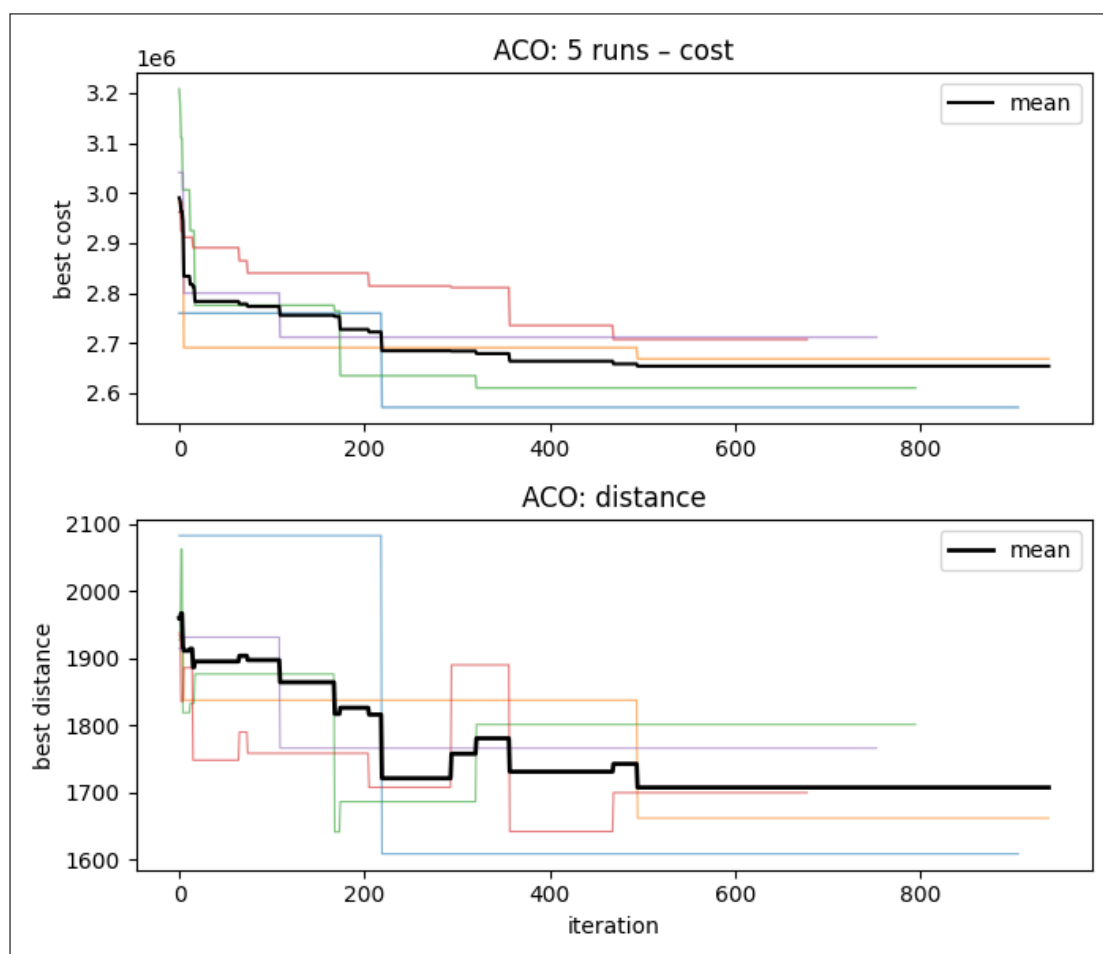


Рис. 1: Графік збіжності 5 запусків АСО

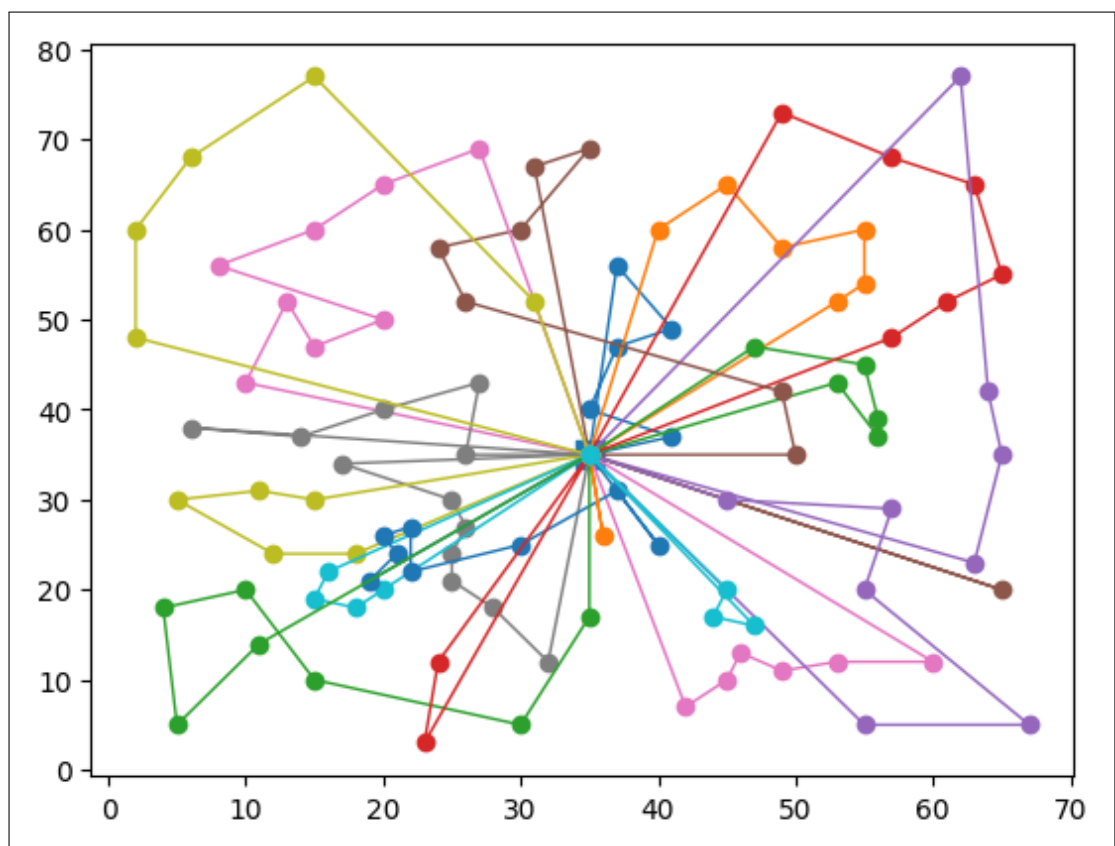


Рис. 2: Графік найкращого маршруту АСО

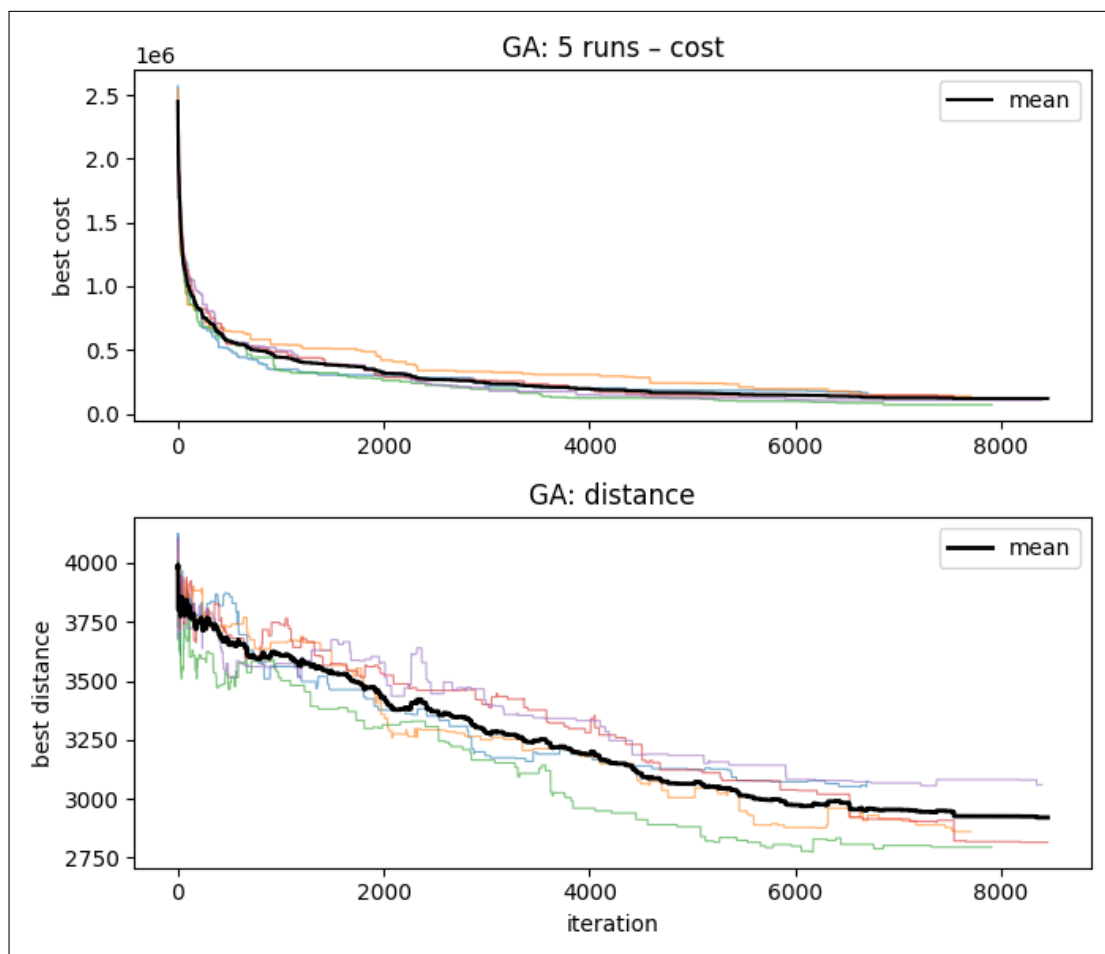


Рис. 3: Графік збіжності 5 запусків GA

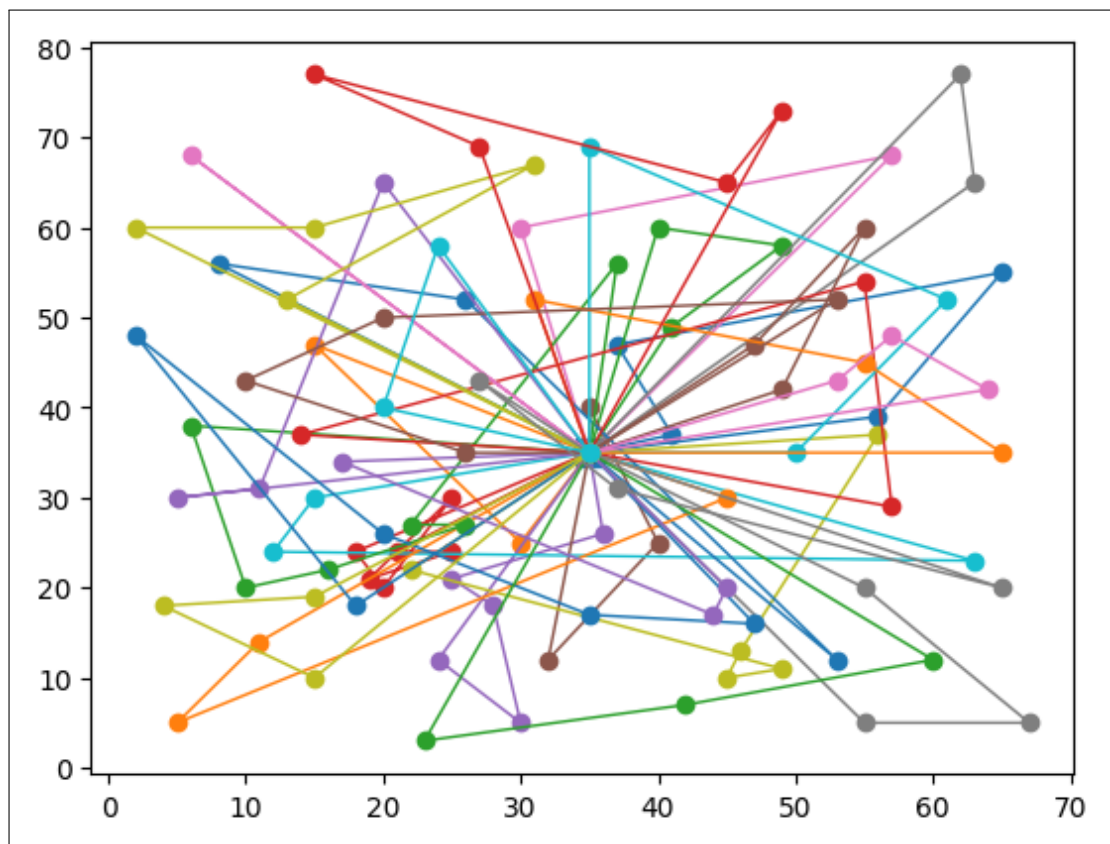


Рис. 4: Графік найкращого маршруту GA

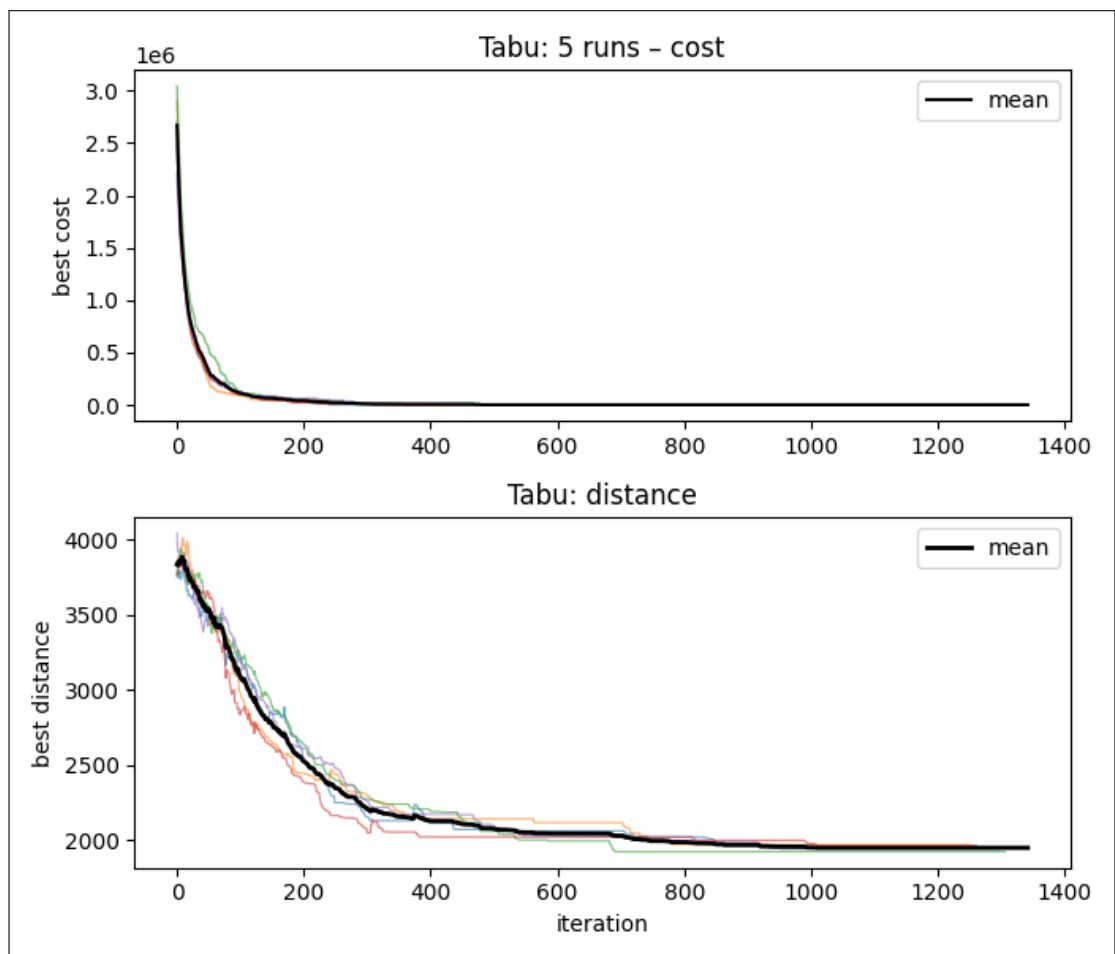


Рис. 5: Графік збіжності 5 запусків Tabu

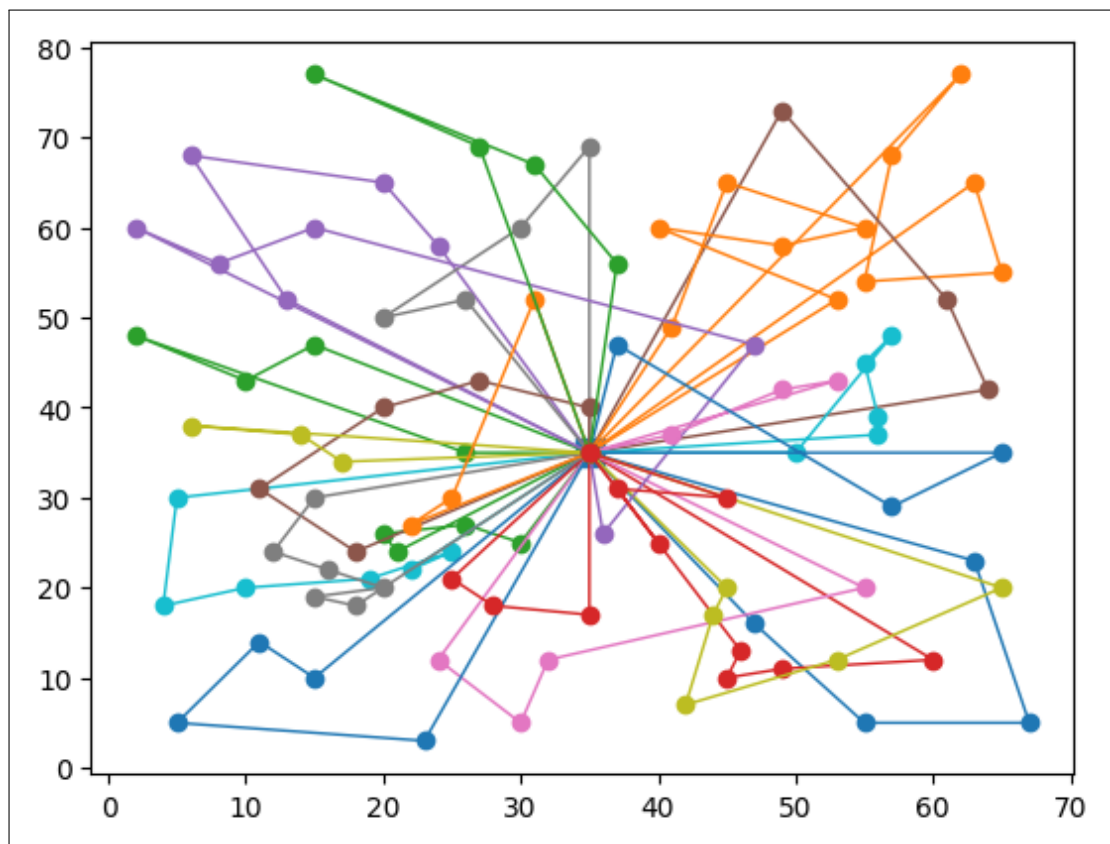


Рис. 6: Графік найкращого маршруту Табу

9. Висновки

1. **Поставлена мета досягнута.** Реалізовано три евристики — мурашиний Ant Colony Optimization (ACO), генетичний GA та траєкторний Tabu Search — з єдиною цільовою функцією та модулем логування; створено скрипт, що автоматично запускає алгоритми, зберігає історію ($iter$, t , $cost$, $dist$) і будує графіки збіжності.
2. **Фізибельність і якість рішень.** За 60 с *лише Tabu Search* у деяких інстансів знаходить повністю фізибельні тури ($cost = dist$); ACO та GA залишають штрафи. Tabu Search демонструє найменший середній $cost$ ($\approx 80\%$ кращий за GA і $\approx 90\%$ — за ACO). ACO дає найкоротшу відстань D , але накопичує великі штрафи; GA без локального пошуку суттєво відстає за обома метриками.
3. **Напрями покращення.**
 - Додавання локального пошуку (2-opt/relocate) до GA та ACO має підвищити якість рішень.
 - Перехід від звичайної ACS до MACS-VRPTW для мурашиного алгоритму очікувано дасть додатковий приріст.
 - Подальші дослідження доцільно спрямувати на автоматичний тюнінг параметрів (α , β , ρ для ACO; $tenure$ — для Tabu) та паралельні GPU/мультипроцесорні реалізації.

Список джерел

1. Dorigo M., Gambardella L.M. Ant Colony System: A Cooperative Learning Approach. *IEEE Trans. Evol. Comput.* 1997.
2. Solomon M.M. Algorithms for the VRPTW. *Operations Research*. 1987.
3. Taillard E., Gambardella L., Agazzi G. MACS-VRPTW. 1999.
4. Laporte G. et al. Handbook of VRP. Springer, 2014.
5. <https://people.idsia.ch/luca/macsvrptw/problems/welcome.htm>